Exercise – 3

Name: Sparsh Karna

Registration Number: 23BDS1172

1. Server

```
import socket

IP = [58,50,42,34,26,18,10,2,60,52,44,36,28,20,12,4,
62,54,46,38,30,22,14,6,64,56,48,40,32,24,16,8,
57,49,41,33,25,17,9,1,59,51,43,35,27,19,11,3,
61,53,45,37,29,21,13,5,63,55,47,39,31,23,15,7]

FP = [40,8,48,16,56,24,64,32,39,7,47,15,55,23,63,31,
38,6,46,14,54,22,62,30,37,5,45,13,53,21,61,29,
36,4,44,12,52,20,60,28,35,3,43,11,51,19,59,27,
34,2,42,10,50,18,58,26,33,1,41,9,49,17,57,25]

E = [32,1,2,3,4,5,4,5,6,7,8,9,8,9,10,11,
12,13,12,13,14,15,16,17,16,17,18,19,20,21,20,21,
22,23,24,25,24,25,26,27,28,29,28,29,30,31,32,1]

P = [16,7,20,21,29,12,28,17,1,15,23,26,5,18,31,10,
2,8,24,14,32,27,3,9,19,13,30,6,22,11,4,25]

PC1 = [57,49,41,33,25,17,9,1,58,50,42,34,26,18,10,2,
59,51,43,35,27,19,11,3,60,52,44,36,
63,55,47,39,31,23,15,7,62,54,46,38,30,22,14,6,
61,53,45,37,29,21,13,5,28,20,12,4]

PC2 = [14,17,11,24,1,5,3,28,15,6,21,10,23,19,12,4,
26,8,16,7,27,20,13,2,41,52,31,37,47,55,30,40,
51,45,33,48,44,49,39,56,34,53,46,42,50,36,29,32]

S = [
[[14,4,13,1,2,15,11,8,3,10,6,12,5,9,0,7],
[0,15,7,4,14,2,13,1,10,6,12,11,9,5,3,8],
[4,1,14,8,13,6,2,11,15,12,9,7,3,10,5,0],
[15,12,8,2,4,9,1,7,5,11,3,14,10,0,6,13]],
# S1
[[15,1,8,14,6,11,3,4,9,7,2,13,12,0,5,10],
[3,13,4,7,15,2,8,14,12,0,1,10,6,9,11,5],
[0,14,7,11,10,4,13,1,5,8,12,6,9,3,2,15],
```

```
[13,8,10,1,3,15,4,2,11,6,7,12,0,5,14,9]],
# S2
[[10,0,9,14,6,3,15,5,1,13,12,7,11,4,2,8],
[13,7,0,9,3,4,6,10,2,8,5,14,12,11,15,1],
[13,6,4,9,8,15,3,0,11,1,2,12,5,10,14,7],
[1,10,13,0,6,9,8,7,4,15,14,3,11,5,2,12]],
# ... (S3 to S7 remain the same)
[[7,13,14,3,0,6,9,10,1,2,8,5,11,12,4,15],
[13,8,11,5,6,15,0,3,4,7,2,12,1,10,14,9],
[10,6,9,0,12,11,7,13,15,1,3,14,5,2,8,4],
[3,15,0,6,10,1,13,8,9,4,5,11,12,7,2,14]],
[[2,12,4,1,7,10,11,6,8,5,3,15,13,0,14,9],
[14,11,2,12,4,7,13,1,5,0,15,10,3,9,8,6],
[4,2,1,11,10,13,7,8,15,9,12,5,6,3,0,14],
[11,8,12,7,1,14,2,13,6,15,0,9,10,4,5,3]],
[[12,1,10,15,9,2,6,8,0,13,3,4,14,7,5,11],
[10,15,4,2,7,12,9,5,6,1,13,14,0,11,3,8],
[9,14,15,5,2,8,12,3,7,0,4,10,1,13,11,6],
[4,3,2,12,9,5,15,10,11,14,1,7,6,0,8,13]],
[[4,11,2,14,15,0,8,13,3,12,9,7,5,10,6,1],
[13,0,11,7,4,9,1,10,14,3,5,12,2,15,8,6],
[1,4,11,13,12,3,7,14,10,15,6,8,0,5,9,2],
[6,11,13,8,1,4,10,7,9,5,0,15,14,2,3,12]],
[[13,2,8,4,6,15,11,1,10,9,3,14,5,0,12,7],
[1,15,13,8,10,3,7,4,12,5,6,11,0,14,9,2],
[7,11,4,1,9,12,14,2,0,6,10,13,15,3,5,8],
[2,1,14,7,4,10,8,13,15,12,9,0,3,5,6,11]]
]

def bin_str(x: int, n: int) -> str:
return f"{x:0{n}b}"

def permute(v: int, p: list, in_bits: int, out_bits: int) -> int:
r = 0
for i in range(out_bits):
bit = (v >> (in_bits - p[i])) & 1
r = (r << 1) | bit
return r

def rotl28(x: int, n: int) -> int:
return ((x << n) | (x >> (28 - n))) & 0x0FFFFFFF

def generate_subkeys(key64: int) -> list:
k56 = permute(key64, PC1, 64, 56)
c = (k56 >> 28) & 0x0FFFFFFF
d = k56 & 0x0FFFFFFF
subkeys = []
shifts = [1,1,2,2,2,2,2,2,1,2,2,2,2,2,2,1]
for i in range(16):
c = rotl28(c, shifts[i])
d = rotl28(d, shifts[i])
cd = (c << 28) | d
k48 = permute(cd, PC2, 56, 48)
```

```
subkeys.append(k48)
return subkeys

def feistel(r: int, k: int) -> int:
e48 = permute(r, E, 32, 48)
x = e48 ^ k
out = 0
for i in range(8):
six = (x >> (42 - 6*i)) & 0x3F
row = ((six >> 5) << 1) | (six & 1)
col = (six >> 1) & 0xF
val = S[i][row][col]
out = (out << 4) | val
return permute(out, P, 32, 32)

def des_encrypt_block(block: int, subkeys: list, debug: list) -> int:
ip = permute(block, IP, 64, 64)
debug.append(f"IP: {ip:016X}")
l = (ip >> 32) & 0xFFFFFFFF
r = ip & 0xFFFFFFFF
for i in range(16):
f = feistel(r, subkeys[i])
nl = r
nr = l ^ f
l, r = nl, nr
debug.append(f"Round {i+1:2d}: {(l<<32|r):016X}")
pre = (r << 32) | l
fp = permute(pre, FP, 64, 64)
debug.append(f"FP: {fp:016X}\n")
return fp

def text_to_block(text: str, pos: int) -> int:
"""Convert 8 characters (or less) to 64-bit block, pad with spaces"""
chunk = text[pos:pos+8].ljust(8)
block = 0
for i, c in enumerate(chunk):
block = (block << 8) | ord(c)
return block

def main():
server = socket.socket(socket.AF_INET, socket.SOCK_STREAM)
server.bind(("localhost", 5001))
server.listen(1)
print("Server listening on port 5000... (waiting for client)")

conn, addr = server.accept()
print(f"Client connected: {addr}")

print("\n=== Encryption side (Server) ===")
message = input("Message/Plaintext: ").strip()
key = input("Key (exactly 8 chars): ").strip()
```

```
if len(key) != 8:
print("Error: Key must be exactly 8 characters!")
conn.close()
return

# Convert key to 64-bit integer
key64 = 0
for c in key:
key64 = (key64 << 8) | ord(c)

subkeys = generate_subkeys(key64)

print("\nIntermediate Results (Encryption):\n")
ciphertext_hex = ""

debug_all = []
i = 0
while i < len(message):
block = text_to_block(message, i)
debug = []
encrypted = des_encrypt_block(block, subkeys, debug)
debug_all.append(f"BLOCK {len(debug_all)+1}:\n" + "\n".join(debug))
ciphertext_hex += f"{encrypted:016X}"
i += 8

print("\n".join(debug_all))
print(f"\nCipher Text (sent to client): {ciphertext_hex}")

# Send ONLY ciphertext to client
conn.send((ciphertext_hex + "\n").encode())

print("\nCiphertext sent to client.")
print("Waiting for client to decrypt...\n")

conn.close()
server.close()

if __name__ == "__main__":
main()
```

2. Client

```
import socket

# DES Tables (same as in Java)
IP = [58,50,42,34,26,18,10,2,60,52,44,36,28,20,12,4,
62,54,46,38,30,22,14,6,64,56,48,40,32,24,16,8,
57,49,41,33,25,17,9,1,59,51,43,35,27,19,11,3,
```

```
61,53,45,37,29,21,13,5,63,55,47,39,31,23,15,7]

FP = [40,8,48,16,56,24,64,32,39,7,47,15,55,23,63,31,
38,6,46,14,54,22,62,30,37,5,45,13,53,21,61,29,
36,4,44,12,52,20,60,28,35,3,43,11,51,19,59,27,
34,2,42,10,50,18,58,26,33,1,41,9,49,17,57,25]

E = [32,1,2,3,4,5,4,5,6,7,8,9,8,9,10,11,
12,13,12,13,14,15,16,17,16,17,18,19,20,21,20,21,
22,23,24,25,24,25,26,27,28,29,28,29,30,31,32,1]

P = [16,7,20,21,29,12,28,17,1,15,23,26,5,18,31,10,
2,8,24,14,32,27,3,9,19,13,30,6,22,11,4,25]

PC1 = [57,49,41,33,25,17,9,1,58,50,42,34,26,18,10,2,
59,51,43,35,27,19,11,3,60,52,44,36,
63,55,47,39,31,23,15,7,62,54,46,38,30,22,14,6,
61,53,45,37,29,21,13,5,28,20,12,4]

PC2 = [14,17,11,24,1,5,3,28,15,6,21,10,23,19,12,4,
26,8,16,7,27,20,13,2,41,52,31,37,47,55,30,40,
51,45,33,48,44,49,39,56,34,53,46,42,50,36,29,32]

S = [
[[14,4,13,1,2,15,11,8,3,10,6,12,5,9,0,7],
[0,15,7,4,14,2,13,1,10,6,12,11,9,5,3,8],
[4,1,14,8,13,6,2,11,15,12,9,7,3,10,5,0],
[15,12,8,2,4,9,1,7,5,11,3,14,10,0,6,13]],
# S1
[[15,1,8,14,6,11,3,4,9,7,2,13,12,0,5,10],
[3,13,4,7,15,2,8,14,12,0,1,10,6,9,11,5],
[0,14,7,11,10,4,13,1,5,8,12,6,9,3,2,15],
[13,8,10,1,3,15,4,2,11,6,7,12,0,5,14,9]],
# S2
[[10,0,9,14,6,3,15,5,1,13,12,7,11,4,2,8],
[13,7,0,9,3,4,6,10,2,8,5,14,12,11,15,1],
[13,6,4,9,8,15,3,0,11,1,2,12,5,10,14,7],
[1,10,13,0,6,9,8,7,4,15,14,3,11,5,2,12]],
# ... (S3 to S7 remain the same)
[[7,13,14,3,0,6,9,10,1,2,8,5,11,12,4,15],
[13,8,11,5,6,15,0,3,4,7,2,12,1,10,14,9],
[10,6,9,0,12,11,7,13,15,1,3,14,5,2,8,4],
[3,15,0,6,10,1,13,8,9,4,5,11,12,7,2,14]],
[[2,12,4,1,7,10,11,6,8,5,3,15,13,0,14,9],
[14,11,2,12,4,7,13,1,5,0,15,10,3,9,8,6],
[4,2,1,11,10,13,7,8,15,9,12,5,6,3,0,14],
[11,8,12,7,1,14,2,13,6,15,0,9,10,4,5,3]],
[[12,1,10,15,9,2,6,8,0,13,3,4,14,7,5,11],
[10,15,4,2,7,12,9,5,6,1,13,14,0,11,3,8],
[9,14,15,5,2,8,12,3,7,0,4,10,1,13,11,6],
[4,3,2,12,9,5,15,10,11,14,1,7,6,0,8,13]],
[[4,11,2,14,15,0,8,13,3,12,9,7,5,10,6,1],
```

```
[13,0,11,7,4,9,1,10,14,3,5,12,2,15,8,6],
[1,4,11,13,12,3,7,14,10,15,6,8,0,5,9,2],
[6,11,13,8,1,4,10,7,9,5,0,15,14,2,3,12]],
[[13,2,8,4,6,15,11,1,10,9,3,14,5,0,12,7],
[1,15,13,8,10,3,7,4,12,5,6,11,0,14,9,2],
[7,11,4,1,9,12,14,2,0,6,10,13,15,3,5,8],
[2,1,14,7,4,10,8,13,15,12,9,0,3,5,6,11]]
]

def bin_str(x: int, n: int) -> str:
return f"{x:0{n}b}"

def permute(v: int, p: list, in_bits: int, out_bits: int) -> int:
r = 0
for i in range(out_bits):
bit = (v >> (in_bits - p[i])) & 1
r = (r << 1) | bit
return r

def rotl28(x: int, n: int) -> int:
return ((x << n) | (x >> (28 - n))) & 0x0FFFFFFF

def generate_subkeys(key64: int) -> list:
k56 = permute(key64, PC1, 64, 56)
c = (k56 >> 28) & 0x0FFFFFFF
d = k56 & 0x0FFFFFFF
subkeys = []
shifts = [1,1,2,2,2,2,2,2,1,2,2,2,2,2,2,1]
for i in range(16):
c = rotl28(c, shifts[i])
d = rotl28(d, shifts[i])
cd = (c << 28) | d
k48 = permute(cd, PC2, 56, 48)
subkeys.append(k48)
return subkeys

def feistel(r: int, k: int) -> int:
e48 = permute(r, E, 32, 48)
x = e48 ^ k
out = 0
for i in range(8):
six = (x >> (42 - 6*i)) & 0x3F
row = ((six >> 5) << 1) | (six & 1)
col = (six >> 1) & 0xF
val = S[i][row][col]
out = (out << 4) | val
return permute(out, P, 32, 32)

def des_decrypt_block(block: int, subkeys: list, debug: list) -> int:
ip = permute(block, IP, 64, 64)
debug.append(f"IP: {ip:016X}")
l = (ip >> 32) & 0xFFFFFFFF
```

```
r = ip & 0xFFFFFFFF
for i in range(16):
f = feistel(r, subkeys[15 - i])
nl = r
nr = l ^ f
l, r = nl, nr
debug.append(f"Round {i+1:2d}: {(l<<32 | r):016X}")
pre = (r << 32) | l
fp = permute(pre, FP, 64, 64)
debug.append(f"FP: {fp:016X}\n")
return fp

def block_to_text(block: int) -> str:
"""Convert 64-bit block back to string, remove padding spaces"""
result = ""
for i in range(8):
byte = (block >> (56 - i*8)) & 0xFF
if byte == 0 or byte == ord(' '):
break
result += chr(byte)
return result

def main():
print("Connecting to server (localhost:5000)...")
client = socket.socket(socket.AF_INET, socket.SOCK_STREAM)
client.connect(("localhost", 5001))

print("Connected! Waiting for ciphertext...\n")

# Receive only ciphertext
ciphertext_hex = client.recv(4096).decode().strip()

print("=== Decryption side (Client) ===")
print("Received Cipher Text:", ciphertext_hex)

key = input("Enter the key (exactly 8 characters): ").strip()

if len(key) != 8:
print("Error: Key must be exactly 8 characters!")
client.close()
return

# Convert key to 64-bit integer
key64 = 0
for c in key:
key64 = (key64 << 8) | ord(c)

subkeys = generate_subkeys(key64)

print("\nIntermediate Results (Decryption):\n")
```

```
plaintext = ""
debug_all = []

for i in range(0, len(ciphertext_hex), 16):
block_hex = ciphertext_hex[i:i+16]
block = int(block_hex, 16)
debug = []
decrypted = des_decrypt_block(block, subkeys, debug)
debug_all.append(f"BLOCK {len(debug_all)+1}:\n" + "\n".join(debug))
plaintext += block_to_text(decrypted)

print("\n".join(debug_all))
print("\nOriginal Plaintext:", plaintext)

client.close()

if __name__ == "__main__":
main()
```

3. Output

```
❯ python3 server.py
Server listening on port 5000... (waiting for client)
Client connected: ('127.0.0.1', 61898)

=== Encryption side (Server) ===
Message/Plaintext: Hello
Key (exactly 8 chars): abcdefgh

Intermediate Results (Encryption):

BLOCK 1:
IP: 1F001E1200FE1D10
Round  1: 00FE1D10401E6151
Round  2: 401E61517AA90FBB
Round  3: 7AA90FBBF59580BD
Round  4: F59580BDA096A683
Round  5: A096A683435DDC65
Round  6: 435DDC65E55421F1
Round  7: E55421F1CCE23F2D
Round  8: CCE23F2DCA1F7180
Round  9: CA1F7180CC93A12E
Round 10: CC93A12E9059F3A7
Round 11: 9059F3A794B3F339
Round 12: 94B3F339B8B60906
Round 13: B8B60906474C05A2
Round 14: 474C05A28AECB444
Round 15: 8AECB44402113C40
Round 16: 02113C40E084293B
FP: 2581180D294D4250


Cipher Text (sent to client): 2581180D294D4250

Ciphertext sent to client.
Waiting for client to decrypt...
```

```
❯ python3 client.py
Connecting to server (localhost:5000)...
Connected! Waiting for ciphertext...

=== Decryption side (Client) ===
Received Cipher Text: 2581180D294D4250
Enter the key (exactly 8 characters): abcdefgh

Intermediate Results (Decryption):

BLOCK 1:
IP: E084293B02113C40
Round  1: 02113C408AECB444
Round  2: 8AECB444474C05A2
Round  3: 474C05A2B8B60906
Round  4: B8B6090694B3F339
Round  5: 94B3F3399059F3A7
Round  6: 9059F3A7CC93A12E
Round  7: CC93A12ECA1F7180
Round  8: CA1F7180CCE23F2D
Round  9: CCE23F2DE55421F1
Round 10: E55421F1435DDC65
Round 11: 435DDC65A096A683
Round 12: A096A683F59580BD
Round 13: F59580BD7AA90FBB
Round 14: 7AA90FBB401E6151
Round 15: 401E615100FE1D10
Round 16: 00FE1D101F001E12
FP: 48656C6C6F202020


Original Plaintext: Hello
```